

Spatial Mass distributions and Galactic Stability

Jonah Cooke

Mar. 5th 2026

Yuan Shi, Plasma Quantum Group

Contents

1. The Problem
2. Whats been tried
3. What I'm trying
4. why I'm trying it
5. the process
6. where Im at
7. Next steps
8. Ideas for future
9. References, Appendix & Credits

The Problem

Via newtonian mechanics and a gaussian integral we expect the velocity distribution of galaxies to look like:

The data shows it follows a curve like:

Whats been Tried

Newtonian physics

$$\mathbf{F} = -\frac{GMm}{r^2}\hat{\mathbf{r}}$$

General Relativistic

$$R^\lambda_{\gamma\alpha}$$

What I am trying

Variable mass:

$$M(r) \propto \sqrt{r}$$

Constant mass:

$$M(r) \propto M$$



Why I am trying it do

- light traveling through plasma

$$\omega^2 = \omega_p^2 + k^2 \iff E^2 = m^2 + p^2$$

- plasma frequency

$$\omega_p^2 = \frac{e^2 n(r)}{\epsilon_0 m}$$

- Since $n = n(r)$, maybe $m = m(r)$

Why I am trying it cont.

$$\omega^2 = \omega_p^2 + k^2$$

Light incident on plasma

- $\Delta E = 0 \Rightarrow \Delta \omega = 0 \Rightarrow -\Delta \omega_p^2 = \Delta k^2$
- In the case $\Delta v < 0 \Rightarrow \Delta \omega_p^2 > 0$
- Since $\omega_p \propto n \Rightarrow \Delta n > 0$
- $\Delta n > 0 \Rightarrow n_1 - n_1 \frac{\sin(\theta_1)}{\sin(\theta_2)} > 0$



Why I am trying it cont.

$$\omega_p^2 = \frac{e^2 n(r)}{\epsilon_0 m}$$

Light incident on plasma

- $n_1 - n_1 \frac{\sin(\theta_1)}{\sin(\theta_2)} = \frac{n_1}{\sin(\theta_2)} (\sin(\theta_2) - \sin(\theta_1))$
- since $\theta_1, \theta_2 \in [0, \frac{\pi}{2}]$ for $\Delta n > 0$,
- $\sin(\theta_2) > \sin(\theta_1)$
- for the domain $[0, \frac{\pi}{2}]$ this implies: $\theta_2 > \theta_1$

The process

$$\mathcal{L} = \mu \frac{\dot{r}^2}{2} + \mu \frac{r^2 \dot{\phi}^2}{2} - U(r)$$
$$\frac{\partial \mathcal{L}}{\partial r} = \frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{r}}, \quad \frac{\partial \mathcal{L}}{\partial \phi} = \frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{\phi}}$$
$$\ddot{r} = -\frac{\partial}{\partial r} U_{eff}(r)$$

The process cont.

$$\mathcal{L} = \mu(r) \frac{\dot{r}^2}{2} + \mu(r) \frac{r^2 \dot{\phi}^2}{2} - U(r)$$

$$\mu(r) \ddot{r} = - \frac{\partial}{\partial r} U_{eff}(r)$$

$$U_{eff}(r) = \frac{\mu(r) \dot{r}^2}{2} + \frac{GM\mu^2(r)}{\mu(r)r^2} - \frac{\ell^2}{2\mu(r)r^2}$$

where I'm at

1. Paired 1-D constant acceleration Lagrangian to a trajectory via a mass distribution:

$$\mathcal{L} = -m(z)\sqrt{1 - \dot{z}^2} \iff z(t) = \frac{1}{a} \left(\sqrt{(at)^2 + 1} - 1 \right)$$

$$m(t) = \frac{1}{\sqrt{(at)^2 + 1}} \iff m(z) = \frac{1}{az + 1}$$

where I'm at cont.

2. Solved the variable mass 2-body central force Lagrangian for EOM's:

$$\mu(r)\ddot{r} = -\frac{\partial}{\partial r} \left[\frac{\mu(r)\dot{r}^2}{2} + \frac{GM\mu^2(r)}{\mu(r)r^2} - \frac{\ell^2}{2\mu(r)r^2} \right]$$

3. Attempted to solve analytically

4. Created python environment to test trial distributions

code

```
import pygame
import math
import os
pygame.init()

width, height = 1200, 800
center_x = width/2
center_y = height/2

G = 1
c = 1
M0 = 40000
EH_R = 10
L = 6
dt = 0.1

TAIL_LENGTH = 300 # number of stored points
TAIL_FADE_DIST = TAIL_LENGTH + 500 # fade scale (distance ahead)

white = (255,255,255)
black = (0,0,0)
orange = (255,165,0)

WIN = pygame.display.set_mode((width,height))
pygame.display.set_caption("planet in GR - fading tail")

def M_of_r(r):
    return M0

def rphi_to_xy(r, phi):
    x = center_x + r*math.cos(phi)
    y = center_y + r*math.sin(phi)
    return x, y

def derivatives(r, phi, vr, omega):
    M = M_of_r(r)
    Fr = -(G*M/r**2)*(1 + (3*L**2/(c**2*r**2)))

    drdt = vr
    dphidt = omega
    dvrdt = r*omega**2 + Fr
    domegadt = -2*vr*omega/r

    return drdt, dphidt, dvrdt, domegadt
```

```
def RK4(r, phi, vr, omega):

    k1 = derivatives(r, phi, vr, omega)

    k2 = derivatives(
        r + k1[0]*dt/2,
        phi + k1[1]*dt/2,
        vr + k1[2]*dt/2,
        omega + k1[3]*dt/2
    )

    k3 = derivatives(
        r + k2[0]*dt/2,
        phi + k2[1]*dt/2,
        vr + k2[2]*dt/2,
        omega + k2[3]*dt/2
    )

    k4 = derivatives(
        r + k3[0]*dt,
        phi + k3[1]*dt,
        vr + k3[2]*dt,
        omega + k3[3]*dt
    )

    dr = dt*(k1[0] + 2*k2[0] + 2*k3[0] + k4[0])/6
    dphi = dt*(k1[1] + 2*k2[1] + 2*k3[1] + k4[1])/6
    dvr = dt*(k1[2] + 2*k2[2] + 2*k3[2] + k4[2])/6
    domega = dt*(k1[3] + 2*k2[3] + 2*k3[3] + k4[3])/6

    return dr, dphi, dvr, domega
```

```
class Planet:
    def __init__(self, r, phi, vr, omega):
        self.r = r
        self.phi = phi
        self.vr = vr
        self.omega = omega
        self.path = []
```

```
def update(self):
    if self.r > EH_R:
        dr, dphi, dvr, domega = RK4(self.r, self.phi, self.vr, self.omega)
        self.r += dr
        self.phi += dphi
        self.vr += dvr
        self.omega += domega

    x, y = rphi_to_xy(self.r, self.phi)
    self.path.append((x,y))

    if len(self.path) > TAIL_LENGTH:
        self.path.pop(0)

def draw(self, win):
    x, y = rphi_to_xy(self.r, self.phi)

    # draw fading tail
    for i in range(1, len(self.path)):
        p1 = self.path[i-1]
        p2 = self.path[i]

        dx = x - p1[0]
        dy = y - p1[1]
        dist = math.sqrt(dx*dx + dy*dy)

        alpha = max(0, 255 * (1 - dist/TAIL_FADE_DIST))
        color = (int(alpha), int(alpha), int(alpha))

        pygame.draw.line(win, color, p1, p2, 2)

    pygame.draw.circle(win, orange, (int(x),int(y)), 5)

class Mass:
    def __init__(self, R):
        self.x = center_x
        self.y = center_y
        self.R = R

    def draw(self, win):
        pygame.draw.circle(win, white, (int(self.x),int(self.y)), int(self.R))

def main():
    os.makedirs("frames", exist_ok=True)
    frame_number = 0
    MAX_FRAMES = 600 # for example
    frames = []
    run = True
    clock = pygame.time.Clock()

    black_hole = Mass(EH_R)

    r0 = 200
    phi0 = 0
    vr0 = -2

    M_init = M_of_r(r0)
    omega0 = math.sqrt((G*M_init/r0**3)*(1 + (3*L**2/(c**2*r0**2))))
```

```
planet = Planet(r0, phi0, vr0, omega0)

while run:
    clock.tick(60)
    WIN.fill(black)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False

    black_hole.draw(WIN)
    planet.update()
    planet.draw(WIN)
    if frame_number < MAX_FRAMES:
        pygame.image.save(WIN, f"frames/frame_{frame_number:04d}.png")
        frame_number += 1
    pygame.display.update()

    pygame.quit()
main()
```



Next steps

1. Find trajectory data.
2. Guess form of $\mu(r)$ based off of data.
3. With analytic form of $\mu(r)$ try to find analytic form of $r(\phi)$
4. Add relativistic corrections.
5. Hone simulation.
6. Apply model to many different systems and check if results agree with observation.

Ideas for future

- How does a variable mass distribution account for Gravitational lensing, and other pieces of evidence supporting dark matter?
- Does this have any implications regarding current quantum gravity frameworks?

References

- 1.



Back to the
beginning